

Crop Pest Early Warning System

Leaf image + weather based pest detection and risk advisory project plan

1. Project overview

The system lets a farmer upload a photo of a crop leaf (rice, wheat, or maize) along with basic weather readings (temperature, humidity, rainfall). The app combines an image-based disease/pest detector with a weather-based risk score to give a final recommendation: what pest or disease is present, how urgent the risk is, and what action to take.

2. How it works (architecture)

- 1 Farmer input: leaf photo + temperature, humidity, rainfall values.
- 2 Image path: a CNN model (trained via transfer learning) classifies the leaf as healthy or a specific pest/disease, with a confidence percentage.
- 3 Weather path: a rule engine applies threshold logic (e.g. high humidity + moderate temperature) to compute a risk level - low, medium, or high.
- 4 Combine step: the pest/disease result and the risk level are merged into one final assessment.
- 5 Output: a recommendation screen showing the detected issue, confidence score, risk level, and the suggested action.

3. Step-by-step build plan

1. Download the dataset

Get the PlantVillage dataset from Kaggle (free account) - it already has labeled healthy and diseased leaf images. Also search for a 'Rice Leaf Disease' dataset for rice-specific images.

2. Set up a coding environment

Use Google Colab (colab.research.google.com) - free GPU access, runs in the browser, nothing to install locally. Upload the dataset there or download it directly inside Colab.

3. Train the CNN model

Use Claude Code (a separate coding assistant tool) to generate the training script. Ask it to build a transfer-learning model using MobileNet on the PlantVillage dataset for leaf disease classification.

4. Save and test the model

After training, the model is saved as a .h5 or .keras file. Test it on a few new leaf photos and confirm accuracy is at least 80-85 percent for a solid demo.

5. Build the weather rule engine

A simple Python function - no machine learning needed. It takes temperature, humidity, and rainfall as input and returns a risk level (low/medium/high) based on agricultural thresholds.

6. Combine logic and build the web app

Use Streamlit (a Python library for building web apps without HTML/CSS). Ask Claude Code to build a Streamlit app that takes the photo and weather inputs, runs both the CNN model and the rule engine, and displays the combined result.

7. Deploy for the demo

Push the code to a GitHub repository, then deploy for free on Streamlit Community Cloud (streamlit.io/cloud). This gives a live link that can be shown directly to the professor.

4. Final output screen

The result screen shows three things clearly: the detected pest/disease name with the model's confidence percentage, a risk level badge (low/medium/high) based on weather conditions, and a one-line recommendation telling the farmer exactly what to do next.

Field	Example value
Detected issue	Blast disease (97% confidence)
Risk level	High
Temperature	28°C
Humidity	86%
Rainfall	Moderate
Recommendation	Spray fungicide within 24 hours - weather favors outbreak spread

Tools used

Task	Tool
Dataset	Kaggle - PlantVillage / Rice Leaf Disease
Model training environment	Google Colab (free GPU)
Code generation	Claude Code
CNN model	Transfer learning (MobileNet)
Weather risk logic	Plain Python (rule-based)
Web app	Streamlit
Deployment	Streamlit Community Cloud